

Binär aritmetik

Med binär aritmetik avses att de involverade talen behandlas som en serie 1:or och 0:or, och manipuleras *bitvis*. En *bit* är en enskild 1:a eller 0:a. I vissa fall är det praktiskt att kunna behandla tal på den lägsta möjliga nivån, d.v.s. bitnivån. Speciellt då man programmerar hårdvarunära applikationer. För mera information om bitvis representation av tal se t.ex. introduktionsmaterial till informationsbehandling eller material om digitalelektronik.

Shiftoperatorerna << och >>

Shiftoperatorerna << och >> används för att shifta den binära representationen av ett tal till vänster (talet * 2 upphöjt till högra argumentet) och till höger (integerdivision med 2 upphöjt till högra argumentet). Några exempel visar vad det är frågan om:

```
10 << 2 == 40
15 << 3 == 90
10 >> 1 == 5
10 >> 2 == 2
```

Att shifta vänsterut med 1 kan ses som ett enkelt sätt att fördubbla ett tals värde.

Varning
Blanda inte ihop shiftoperatorerna << och >> med I/O-operatorerna med samma namn som C++ använder för att läsa ock skriva streams! Om du använder shiftoperatorer i samband med I/O bör du för att undvika fel använda parenteser för att försäkra dig om att shiftningarna utförs före eventuell I/O. För mera info om C++ I/O-funktioner se Kapitel 8 .

Bitvisa operatorerna |, &, ^ och ~.

Dessa fyra operatorer kan användas för att manipulera enskilda bitar i ett tal. Detta kan behövas om t.ex. något standard bibliotek råkar använda sig av enskilda bitar i någon parameter för att representera någon viss funktionalitet, eller när man programmerar hårdvarunära och behöver ändra en viss bit i ett register.

För att göra en viss bit i ett tal till 1 kan man använda sig av operatoren |. Detta är en vanlig binär *or*. För att ändra läge på en viss bit kan man använda sig av operatoren ^, som fungerar som en normal *exclusive or*. Att "maska" ut vissa bitar ur ett tal kan man göra med &, som är en normal binär *and*. Sist finns operatoren ~ som är en binär *not*, och inverterar ett tals binära representation. Några exempel visar hur dessa operatorer fungerar:

```
// sätta en viss bit eller flera bitar
int Resultat = Tal | bitmask;
int Resultat = Tal | 64;

// ändra värde på en viss bit
int Resultat = Tal ^ bitmask;
int Resultat = Tal ^ 32;
```

```
// maska ut vissa tal, t.ex. de lägsta 8 bitarna
int Resultat = Tal & bitmask;
int Resultat = Tal & 255;

// nollställa en viss bit (negering först)
int Resultat = Tal & ~bitmask;
int Resultat = Tal & ~8;
```

Det sista exemplet kan behöva lite förklaring. För att nollställa en viss bit inverteras först den bitmask som man vill nollställa. Därefter görs en *and* med originaltalet. Detta har som resultat att endast den biten som skall nollställas är 0 i masken, och enligt hur *and* fungerar blir resultatet då det ursprungliga talet med den ena biten (eller flera bitar, om så önskas) nollställd.

[Förutgående](#)

Mera om division

[Hem](#)[Upp](#)[Nästa](#)Inkrementerings- och
dekrementeringsoperatorerna ++
och □